Computers & Security 148 (2025) 104171

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Thread-sensitive fuzzing for concurrency bug detection

Jingwen Zhao, Yan Fu, Yanxia Wu *, Jibin Dong, Ruize Hong

College of Computer Science and Technology, Harbin Engineering University, Harbin, 150001, Nantong Street, China

ARTICLE INFO

Keywords:
Fuzzing
Concurrency bugs
Bug detection
Thread interleaving

ABSTRACT

Fuzzing is a commonly used method for identifying bugs and vulnerabilities in software. However, current methods for improving fuzzing in concurrency environments often lack a detailed analysis of the program's concurrent state space. This leads to inefficient execution of previously verified concurrent states and missed information. We have developed TSAFL, a novel concurrency fuzzing framework that aims to detect the running state of concurrency programs and uncover hard-to-find vulnerabilities. TSAFL builds upon AFL's concurrency vulnerability detection capabilities by incorporating three new techniques. Firstly, we introduce two new coverage metrics to measure concurrency: concurrent behavior window and CFG prediction. These metrics enhance the TSAFL's capabilities to explore more thread interleavings. The second technique adds efficient thread-interleaved scheduling to fuzzing combined with period scheduling. Several methods are proposed to avoid problems caused by simply using period scheduling to accurately detect and verify all concurrent state spaces. Thirdly, we propose a multi-objective optimization mechanism based on the characteristics of concurrent fuzz testing to fully utilize the information in the seed files. Using these three techniques, our concurrency fuzzing approach effectively covers infrequent thread interleavings with concrete context information. We evaluated TSAFL on user-level applications, and experiments show that TSAFL outperforms AFL++ and MOPT in multithreading-related seed generation and concurrent vulnerability detection.

1. Introduction

Concurrent programs have been popular in the current multi-core processor era since they substantially utilize hardware resources to boost software performance. Concurrency issues become more prominent with increasing scale and different applications using concurrency to improve efficiency (Arifin and Atika, 2021). Compared to sequential programs, however, concurrent programs are more prone to serious software failures. On the one hand, the non-deterministic thread interleaving gives rise to concurrency bugs like atomicity violations, data races, etc. This non-deterministic interleaving makes it difficult to reproduce result anomalies or bugs, leading to serious safety issues (Lu et al., 2008). On the other hand, specific inputs and thread executions may lead to concurrency bugs, resulting in system crashes, such as memory corruptions, assertion failures, etc (Cai et al., 2019). Therefore, how to effectively detect these concurrency bugs becomes crucial.

To find concurrency bugs, it is essential to intervene in thread interleaving by controlling thread scheduling. Each concurrency bug requires a specific interleaving order and input to trigger it, we need to deallocate memory before using it, and an input that creates multiple threads of execution. However, it faces the challenge of not being able to efficiently explore concurrent programs and wasting resources. Due to the non-deterministic and random nature of concurrent programs, it is not possible to fully explore all scenarios of thread interleaving and there is no guarantee that the program under test will run as expected. If a bug is triggered by an infrequent thread interleaving, the bug is difficult to detect. Meanwhile, in the current context, where most operating systems use time slicing and other means of resource allocation, if neighboring concurrent threads are extremely close together, the only way to control the program is through near-blind multiple executions, resulting in a huge waste of time resources.

There exists a body of works (Hong and Kim, 2015) on detecting bugs in concurrent programs. Static analysis (Barbar and Sui, 2021; Voung et al., 2007) explores program context-sensitive correlations and detects potential bugs in concurrent programs without actual execution. However, it generates many false positives to ensure thorough exploration and requires additional steps to validate detected bugs. Dynamic detection (Ahmad et al., 2021; Kidd et al., 2011; Lu et al., 2011) monitors the program execution and provides high accuracy, but is limited by test cases. However, these methods have limitations and by themselves do not automatically generate new test inputs to execute different paths in concurrent programs. It requires a large number of test cases to activate the interleaving of different defect paths.

* Corresponding author.
E-mail address: wuyanxia@hrbeu.edu.cn (Y. Wu).

https://doi.org/10.1016/j.cose.2024.104171
Received 8 July 2024; Received in revised form 16 September 2024; Accepted 19 October 2024
Available online 28 October 2024

Fuzzing generates (Böhme et al., 2020; Li et al., 2018; Zhu et al., 2022) more efficient test cases at runtime and explores a program's execution path to find undiscovered bugs, making it a promising approach to overcome other detection methods' limitations. AFL++ (Fioraldi et al., 2020) triggers different code paths through many generated test cases, thus improving code coverage and the probability of triggering crash. However, since the approach primarily observes the behaviors triggered by the characteristics shared over the majority of seed inputs, they ignore the impact of different individual characteristics of each seed input. Meanwhile, general-purpose fuzzy testing tools (Peng et al., 2018; Rawat et al., 2017; Inc., 2019) are limited in exploring thread interleavings by using sequential code coverage as program feedback and neglecting thread interleavings.

To improve the ability of fuzzy testing to find concurrency bugs, researchers (Ko et al., 2022; Lyu et al., 2019; Jiang et al., 2022; Chen et al., 2020) have used information about the running behavior of concurrent programs to perform thread interleaving exploration and enhance the sensitivity of concurrent programs. MUZZ (Chen et al., 2020) relies on intermittent schedule control, which selects multithread-related seeds by distinguishing execution states in multi-threaded contexts. However, it use a context-insensitive random thread-interleaving exploration. MOPT (Lyu et al., 2019) adaptively finds the optimal mutation strategy through a learning algorithm based on particle swarm optimization, but it does not capture individual characteristics of seed inputs for different seed inputs. So, they may miss many deep hidden data races that occur only in specific runtime contexts. Conzzer (Jiang et al., 2022) and AutoInter-fuzzing (Ko et al., 2022) by using execution statistics such as function call pairs and concurrent memory access pairs, which in turn explore the state space. However, they generate a large interleaved space during exploration. Meanwhile, relying exclusively on the concurrent behavioral information of the running process, ignoring the influence of other parts, can lead to performance loss.

Therefore, how to introduce the concurrency dimension in fuzzy testing to more effectively test concurrent programs and detect bugs or vulnerabilities in them is a question we need to consider. To this end, we propose a dedicated fuzzy testing tool, TSAFL, based on AFL (Zalewski, 2020), which can detect concurrency bugs by exploring the state space of concurrent programs in more detail. First, the approach uses information collected during static analysis and program execution as fuzzy feedback. It introduces two new metrics to improve the mining capabilities: concurrency behavior window and CFG prediction. These metrics accurately reflect the state of program execution, provide contextual information for program evaluation, and explore as many thread interleavings as possible. The CFG prediction also provides richer information for fuzzy testing based on control flow graphs, making the process more efficient.

In the fuzzy testing phase, the component uses the information from the previous phase to store the inputs that trigger concurrent bugs, which are divided into two phases: exploration and exploitation. We propose a multi-objective optimization strategy that can balance the value proposition of the testing direction among different metrics. During exploration, TSAFL statistically analyses the data from all seed files to establish the relationship between different index detection parameters, which provides promising directions for thread interleaving exploration. In the exploitation part, the energy distribution is adjusted based on the overall performance of the seed files, and the differences between the seed files are used to apply targeted mutation operations to effectively cover different thread interleavings with specific contextual information. Meanwhile, to address the lack of thread space exploration in traditional fuzzy testing, TSAFL combines an optimized period scheduling strategy with a new thread interleaving scheduling module to progressively explore the scheduling space with the goal of feasible interleaving. The module accurately schedules threads from a holistic perspective and improves the efficiency of exploring the program state space.

```
void foo(bool ifInc){
    pthread_t tid;
    int arg = 0;
    threas_create(&tid, write, &arg);
    if (ifInc);
       arg++;
    threas_join(tid);
    arg++;
}
void* writer(void *arg){
     write(arg);
}
```

Fig. 1. A data race case.

Benefiting from the different thread interleavings and related contextual information identified by the method, the tool can efficiently and accurately detect concurrency bugs in real programs. We have implemented TSAFL and evaluated its effectiveness on seven real programs in comparison with AFL++ and MOPT. The experimental results show that TSAFL outperforms AFL++ and MOPT in detecting concurrency bugs, finding 7 data races.

To improve current concurrency fuzzing techniques, we propose a new approach for fuzz testing that can more effectively explore the state space of concurrency programs. Overall, we make the following main contributions:

1. We developed a concurrency fuzzing test tool called TSAFL to efficiently explore thread interleaving and detect concurrency bugs. By introducing two new metrics to improve context sensitivity in concurrency coverage: the concurrent behavior window and CFG prediction.
2. We optimize seed selection and execution decisions by integrating a multi-objective optimization strategy to guide seed file energy allocation, which helps to generate more concurrent context-sensitive seeds.
3. We have implemented a new thread interleaved scheduling method for fuzzing tests based on period scheduling to improve the efficiency of program state space exploration.

This paper is organized as follows. We briefly describe the background of bugs in concurrency programs and fuzzing tests in Section 2. An overview of the design is provided in Section 3. Section 4 presents the static analysis of TSAFL and Section 5 describes the fuzzing with elaborated thread interleaving. The evaluation results are provided in Section 6. The related work is presented in Section 7.

2. Background

2.1. Bugs in concurrency programs

Concurrency bugs can be challenging to detect because concurrent programs are complicated (Fu et al., 2018). This complexity can make it difficult for developers to even realize that these bugs exist. Concurrency bugs usually occur when multiple threads run concurrently and affect other threads.

Fig. 1 shows an example of a typical concurrency bug - data race. The function `foo` creates a thread to execute the function `writer`. There could be a data race when the variable `arg` is incremented more than once because the function `writer` and the `arg++` instruction can be executed in parallel. When two different threads simultaneously access the same shared data for read and write operations, and at least one of them is a write operation, a data race occurs (Yu et al., 2019).

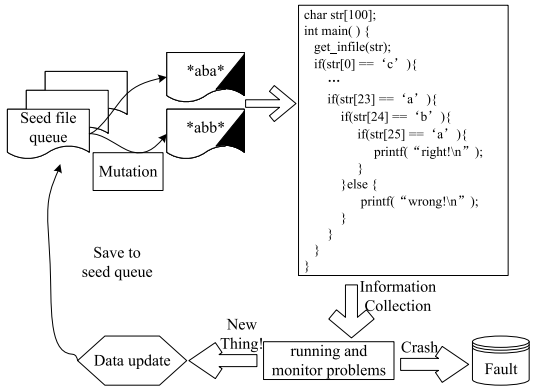

Fig. 2. Workflow of mutation-based greybox fuzzer.

Two main factors can cause these bugs during program execution: program input and thread scheduling. In a sequential program, the program's execution and results are mainly determined by its input. This forms the basis for most program testing, which helps to ensure program correctness and uncover various bugs. On the other hand, in a concurrent program, the program's execution is influenced not only by input but also by the impact of thread scheduling that occurs randomly during runtime. Thread scheduling provides increased flexibility for writing concurrent programs and is also an essential part of ensuring system timeliness. However, this also means that each execution of a concurrent program may be influenced by time, resulting in different running states, or producing different execution results due to the instability of thread execution order. Therefore, concurrency defects caused by thread scheduling have become a special type of defect in concurrent programs, which are challenging to detect and have a high degree of harm.

2.2. Grey-box fuzzing test

In recent years, researchers have started using fuzzing technology (Chen and Chen, 2018; Nguyen et al., 2020; Böhme et al., 2016) to detect concurrency bugs. Grey box fuzzy testing based on mutation is a type of fuzzy testing that uses lightweight instrumentation to extract coverage information such as path and code coverage to determine the quality of test cases. As shown in Fig. 2, it generates a large number of test cases through seed selection, seed mutation, and execution. By repeating these three steps, it can automatically test the target program with numerous different test cases to explore deeper program locations and detect vulnerabilities.

The importance of seed selection for fuzzy testing cannot be overstated, it determines the direction of fuzzy testing as a whole. The selection method and strategy determine what kind of test files the fuzzy testing is sensitive to and what kind of test files it chooses to store and use as part of future seed files.

Seed mutation operations refer to the use of strategies to make various types of changes to the seed file based on the original seed file. Examples of mutation strategies are bitflip, arithmetic, interest, dictionary, havoc, and splice. However, the traditional seed mutation operation is almost blind, not based on a clear knowledge of the seed file. Instead, the mutation operation is performed on each seed using the same strategy, ignoring the information that the seed file itself possesses will lead to a waste of information resources.

Meanwhile, fuzzy testing tools use a large number of different coverage metrics in practice. Currently, the most mainstream fuzzy testing focuses on code coverage, such as critical node coverage, basic block (BB) coverage, and so on. Suppose the current set of seed queues C consists of two seeds s_1 and s_2 that cover the following addresses of the PUT: $\{s_1 \rightarrow \{10, 20\}, s_2 \rightarrow \{20, 30\}\}$. If we have a third seed $s_3 \rightarrow \{10, 20, 30\}$ that executes roughly as fast as s_1 and s_2, one might discuss that seeding file s_3 makes more sense than s_1 and s_2 because it intuitively tests more program logic at half the cost of execution time. The higher the code coverage, the greater the chance of finding bugs.

2.3. Challenges of concurrent fuzz testing

In order to reveal vulnerabilities in concurrent programs, GBF needs to identify the running state of multithreaded programs and generate different seeds. However, due to the use of code coverage as program feedback, existing GBFs are unable to correctly distinguish whether new meaningful seed files are generated during concurrent program runs. To improve fuzzy testing for detecting concurrent programs, concurrent fuzzy testing methods have been proposed, but the following challenges still exist:

Since obtaining thread interleaving information can be difficult, many approaches attempt to explore thread interleaving through static or dynamic conditional thread prioritization, e.g., random delays (Xu et al., 2020), and wringing taps methods (Liu et al., 2018; Chen et al., 2018). They all rely heavily on the program under test characteristics and randomness, and there is no guarantee that the program under test will run with the same results as the expected plan. If adjacent concurrent behavior is very close, in the context of the vast majority of current operating systems that utilize time slices and other means of thread time resource allocation, it is extremely difficult to accurately control the program by means of adjusting thread priority, which often duplicates thread interleavings that have already been covered and misses many uncommon thread interleavings.

Also, different seed files need to be generated to execute different paths in a multi-threaded context. Relying solely on code coverage or concurrent pairs as a coverage metric is incomplete. KRACE (Xu et al., 2020) proposes an alias instruction pair that only describes the location of two concurrently-executed instructions and ignores their calling contexts, so the metric cannot differentiate concurrently-executed situations in different calling contexts. Also, it injects random delays at memory access points, which can frequently repeat already-covered thread interleavings and miss many infrequent thread interleavings. In addition, performing random thread interleaving exploration may miss certain program operations. Jeong et al. (2019) uses custom hypervisors to proactively generate system calls and control their thread interleavings to explore and detect data race in the Linux kernel. MUZZ (Chen et al., 2020) randomly adjusts thread priorities at runtime and selects multithread-related seeds by distinguishing execution states in multithreaded contexts. Their reliance on intermittent or loose control of the schedule (e.g., delayed injection) may make it difficult or impossible to reach some interleavings in the test environment itself, making specific thread interleavings infrequent in coverage.

3. Overview

TSAFL (Thread Sensitive AFL) is a fuzzing tool that is sensitive to threads. It is an extension of AFL and runs on a Linux system. The tool was developed to address the fact that current fuzz testing primarily focuses on improving code coverage while disregarding the significance of concurrency interleaving in multi-threaded programs. As shown in Fig. 3, TSAFL uses static analysis results to enhance fuzz testing performance by improving information collection and seed selection. It is divided into two parts: the state space exploration system (Section 4) and the fuzzy testing implementation system (Section 5).

The information collection module comprises two parts: a static analysis module and a dynamic collection module. The purpose of the static analysis module is to find all sensitive operations that access the same memory address in different threads. We use SVF (Sui and Xue, 2016) and LLVM (llvm, 2023) to analyze the LLVM-IR code of the source program and obtain information related to the position of concurrency-sensitive behavior (Section 4.1). Meanwhile, TSAFL uses

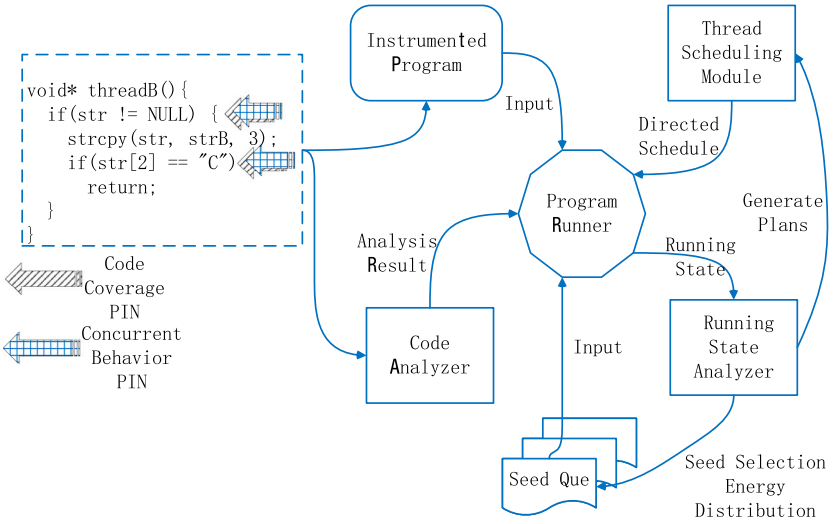

Fig. 3. TSAFL implement.

LLVM-PASS to obtain control flow graph information for the source program and outputs the distance file.

The dynamic collection module consists of a PIN program (Cur-clang) and instrumented code. Cur-clang is a pre-packaged Clang compiler that can perform PINs during the compilation process to insert the required code into the corresponding location with the assistance of the provided location file. The instrumented code has two components: concurrent behavior window information collection and thread scheduling. The information collection component (Section 4.2) records code coverage and concurrency-sensitive information, including thread creation or join, execution time, and concurrency-sensitive behavior sequence. Scheduling scenarios are guided through the concurrent behavior window (Section 4.3.1) to help correctly understand the state of the program and mitigate combinatorial deficiencies in scheduling.

Following static analysis and dynamic collection, TSAFL uses the collected information to run fuzzy testing, including the thread scheduling module, run state analysis module, seed selection, and energy distribution module. During fuzz testing, TSAFL optimizes seed selection, energy distribution, and thread scheduling to generate more concurrency-relevant seeds, effectively exploring thread interleaving by identifying and overriding operations. It differs from traditional mutation-based fuzzy testing tools in several important aspects, as shown in Fig. 3. First, the new seed case is added to the schedule execution and analysis system after evaluating the test case using three different metrics with the feedback analyzer. After the seed file is added, the thread scheduling module generates scheduling plans based on the collected information and performs thread interleaving scheduling on the program under test. Second, the energy distribution module changes the mutation energy before the seed files are added to a new turn based on their performance of different metrics compared to the entire seed queue. Lastly, when the time is right, TSAFL calls the MTM module to perform targeted mutations on the seed file to improve the next generation of inputs at runtime. When a crash occurs, the fuzzer records the test case and the execution order of the operations used to execute the target program.

4. Concurrent state space exploration

The acquisition and utilization of information in defect detection is one of the key factors in determining the detection effect. The component includes two parts: static analysis and dynamic information collection. The static analysis extracts the location information of the program through code coverage, concurrency-sensitive operations, and CFG information. Dynamic analysis uses staking functions to extract the dynamic runtime information of the program. Meanwhile, new concurrency behavior recording methods and prediction exploration methods are introduced.

4.1. Concurrency-sensitive operations

We need to extract concurrency -sensitive operations to find the state space to be explored because not all thread interleavings create concurrency defects. We aim to detect concurrency vulnerabilities such as concurrency defects (data race), double-free, stack-overflow, and so on. In concurrent programs, uncertain accesses to regions of shared variables between threads may make the program buggy. Therefore, we define concurrency-sensitive operations that may affect the behavior of other threads and thus trigger concurrency bugs. The following lists the possible types of concurrency-sensitive operations:

1. Read/Write operations accessing shared memory and global variables.
2. Thread-specific function calls: such as calling external functions with pointer arguments.
3. Related sensitive operations include various types of `mutex` `lock/unlock`, `spinlock`, read/write lock `rwlock`, and condition variables.

Fig. 4 shows a code segment abstracted from a real program that assigns jobs for a certain input file to various threads. It has similar behavior to `pbzip2`, `libvpx`, and others. After reading the environment variable, it will operate on a pointer common to both threads based on the environment variable, and a different thread will perform a free operation on the pointer after checking the common pointer. The multithreaded context starts with `pthread_create`, which contains the shared variables `g_pointer` and `env`, both of which are passed through the global variables as well as the check and free operations performed on the shared variables. Shading indicates recognized concurrency-sensitive operations.

4.2. Information collections

During program execution, we identify and record the following information that may cause concurrency bugs:

- Concurrency-related function evocations and locations. This includes operations such as thread creation, invocation, destruction, abort, and thread cycle scheduling.

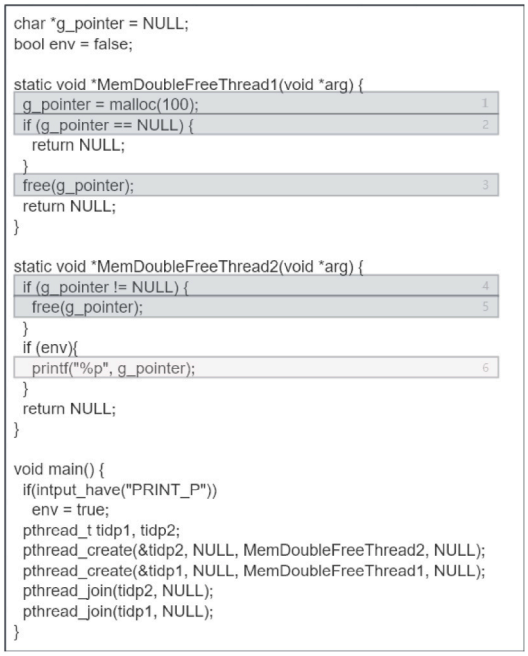

Fig. 4. Application of code instrumentation technology.

- Concurrency-sensitive operation information and location. This includes operation such as function call, lock acquisition/release, shared variables that may trigger data race.
- The combination of instruction characteristics divided by functions. This includes the location and type of the instruction, the numbers and types of the associated variables, and the special flags for the different types of instructions (For example, reading variables and assigning values to variables).

For identifying these accesses before fuzzing, we implemented a static analysis using a custom LLVM-PASS and SVF to automatically analyze program code. We fully utilized SVF's data flow and pointer analysis tools, combined with Andersen's algorithm to obtain concurrency-related information, to determine the location of concurrency-sensitive behaviors and the correlation between different concurrency-sensitive behaviors. Follow the form of {(*variable name* : (*operation type*, *operationtype location*)...), ...} to store the source code information. Meanwhile, effectively filter the above information, and ignore irrelevant variables and special symbols and so on.

To diversify the thread interleaving, we use code instrumentation techniques to collect dynamic information about program execution. Compared to the regular approach of code-coverage oriented instrumentation, we apply a combination of concurrency-sensitive operations to distinguish thread identities for additional feedback. This complements coverage-oriented instrumentation, which is unaware of thread IDs. Meanwhile, to meet the requirements of thread scheduling, it is necessary to insert thread cycle management code at the same location. The form of instrumentation functions can improve the adaptability of the tool and avoid scenario problems caused by a lack of header files and other issues. This instrumentation is general enough to be applied to different concurrent programs and extremely lightweight to keep the runtime overhead minimal.

Fig. 5 shows an example of applying the instrumented technique, where the TSAFL collects the corresponding scheduling point instructions, and the instrumentation assigns a context-aware ID(`loc1,loc2`) to each thread and operation. In this case, the code instrumentation functions are inserted at locations corresponding to concurrency-sensitive behaviors, and the code coverage instrumentations are inserted in the form of code blocks.

4.3. The concurrent program running metrics

To improve the ability of fuzzy testing to detect multithreading, so that the new seed selection strategy is better adapted to the collected concurrent program information. We redefine the test file value metrics to more effectively explore the operational state space and concurrency vulnerabilities embedded in concurrent programs through inter-thread concurrency behavior interleaving information, code coverage information, and CFG (control flow graph) information.

4.3.1. Concurrent behavior window

When considering a program's memory space as a combination of different nodes, such as sensitive behavior pairs, the collection of serial programs remains relatively stable across multiple executions. However, the components of multi-threaded or concurrent programs vary significantly between executions, and recording using code coverage does not encapsulate the contextual information of a concurrent program. The frequency distribution of different running state space features is also not balanced, with some features occurring only rarely.

To accommodate the characteristics of concurrent programs, we propose a new metric `concurrent behavior window` that considers the context information of the concurrent state space:

$$ConBeWin = \{(Thread_1, NodeInfo_1), \ldots, (Thread_2, NodeInfo_x), \ldots, (Thread_1, NodeInfo_n)\} \quad (1)$$

We describe the runtime state space nodes of a concurrent program using $NodeInfo$, which contains contextual information about each instruction in the thread interleaving, including the location of the function call ($NodeLoc$), and the potentially concurrency-sensitive operations ($NodeOpera$). Specifically, we describe a node containing potentially concurrent behavior as:

$$NodeInfo_x = [NodeLoc, NodeOpera] \quad (2)$$

As shown in Fig. 6, the potential concurrent behavior of two adjacent threads of the same thread as the two ends of the window. The concurrent behavior of different threads is considered as potential window intruder, and the interruption to continuous thread operations are considered as potential anomalies. To simplify the analysis, we only consider the relationship between the two threads here.

Fig. 7 presents the concurrent behavior window operation process for the code shown in Fig. 4, which generates multiple possible thread interleaving results and contextual information. The concurrent behavior window identifies runtime information about threads `Thread1` and `Thread2`, and selects concurrency-sensitive operation locations. Then, the window checks the information about `Thread2`, treats the two neighboring nodes 4 and 5 as the two ends of the window, and infers that 4 and 5 could be also concurrent with `Thread1` when they are executed in specific calling contexts. Based on this, the window combines 4 and 5 with the concurrency-sensitive operations in Thread1 to generate four new possible concurrently executing processes. Similarly, the window also handles combinations of other nodes in `Thread2` with concurrency-sensitive operations in `Thread1`, and combinations of neighboring nodes in `Thread1` with concurrency-sensitive operations in `Thread2`, thus generating more state spaces with contextual information.

In general, the concurrent programs often need to be executed multiple times to obtain all pairs of concurrent behaviors, and all concurrent behavior pairs do not mean that all state-space situations have been traversed. The concurrent behavior window considers more concurrency-sensitive behaviors at the same time, which is closely connected with the reasons why concurrency defects occur. Bad thread interleaving means breaking the sequence of sensitive behaviors that should be continuous from the perspective of a single thread. The concurrent behavior window can fully express such a situation.

At the same time, the advantages brought by concurrent behavior windows are as follows:

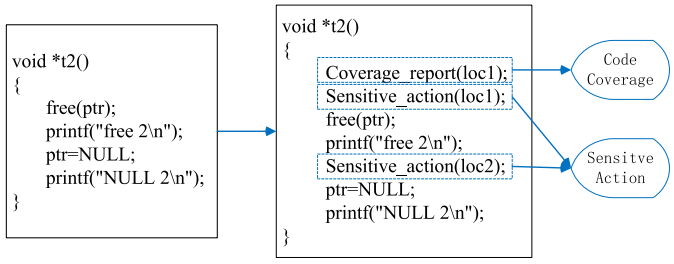

Fig. 5. Application of code instrumentation technology.

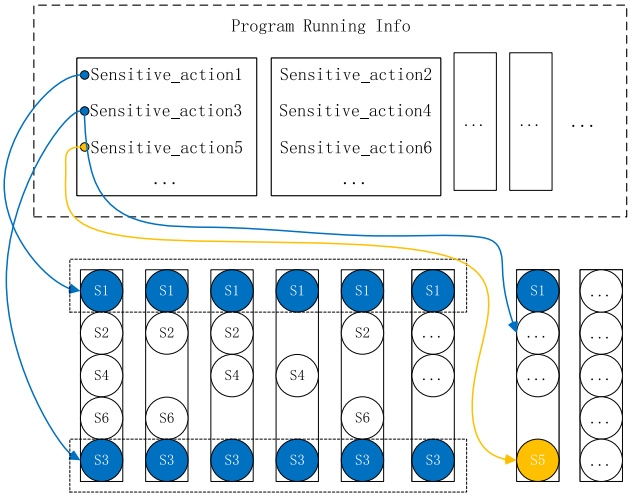

Fig. 6. Concurrent behavior window diagram.

(1) Adapt to a more in-depth scheduling algorithm. We will introduce a new scheduling algorithm that can adjust the progress of each thread more deeply, which brings about the problem of space explosion. The concurrent behavior window can help reduce the number of scheduling schemes. At the same time, because the concurrent behavior window has the characteristics of starting from one thread and returning to the same thread, it can reduce the lack of combination caused by simplifying the scheduling scheme.
(2) Bearer thread interleaving scheduling results. We use the Period Scheduler (Section 5.2.1) to create and cooperate with program instrumentation to execute scheduling schemes. Since the scheduling scheme is created based on static analysis, it is easy to have an invalid scheduling scheme due to non-detailed static analysis results. As shown in Fig. 8, S4 fails in actual execution. In this case, by comparing the predicted results with the actual execution results, it is possible to complement and correct the concurrency-related information obtained and collated by the static analysis. Based on this, the information obtained from analyzing the concurrency behavior window can guide the periodical scheduling to generate the corresponding execution plan more efficiently. At the same time, we directly delete plans with the same structure in the concurrent behavior window.

4.3.2. CFG prediction

The CFG illustrates potential contextual information between different nodes on the execution path–most importantly, the portion adjacent to the actual execution path. These potential paths exist due to the different program flow directions caused by various branch nodes in the control flow chart, such as different tributaries affected by switch judgment conditions. As illustrated in Fig. 9, analyzing the data in CFG indicates that the flow of execution on the left can be altered using branches. Therefore, it can be concluded that fuzz testing can purposely adjust the focus and direction of testing by obtaining predictive information through the potential execution paths contained in the statistical CFG.

The CFG also contains information related to concurrent behavior, which can be an important basis for measuring the value of concurrent program test files. We have established current state-based CFG prediction metrics to select higher value seed files by comparing the number and distance of potential thread intersection opportunities for different seed files. The basic idea of obtaining an effective metric is to conjunction the potential number of paths and distance dependencies associated with that seed file. The formalization is described as:

$$PM(CFG) = CB(a) \wedge \sum PP(a,b) \wedge DD(a,b) \tag{3}$$

As shown in the above equation, the CFG prediction metrics include following parts:

(1) $CB(a)$ is a behavioral judgement. It indicates that the CFG node a includes concurrency-sensitive behaviors.
(2) $PP(a,b)$ is the number of potential paths from node a to node b. Some potential paths, although containing concurrency-sensitive operations, should not be taken seriously if they no longer provide new opportunities for thread interleaving.
(3) $DD(a,b)$ is the distance dependency from node a to node b. If a longer distance in the CFG means that the program needs to perform more branch crossings, unknown test files with farther branch distances are more difficult to generate through the current seed file.

However, since the CFG is a huge source of information, its use needs to be strictly limited to avoid adding a time burden to the fuzzing process due to the analysis process alone. Therefore, concurrent fuzzing follows the three principles of single-layer depth, concurrency sensitivity, and limited combination.

(1) The principle of single-level depth only considers program paths that can be reached through a single-branch transition (including all potential branches in multi-branch) in the current running path.
(2) Concurrency sensitivity means that nodes unrelated to concurrency will be ignored to reduce the complexity of path calculation and improve detection efficiency without losing the effect of concurrent detection.
(3) The finite combination principle is for multi-threaded situations, where the potential value of different threads is only estimated within the thread when evaluating, and the interaction relationship between different threads is no longer considered.

5. Dynamic Fuzzy implementation

In the fuzzy testing step, we aim to detect concurrency bugs by monitoring program runs, and concurrency-sensitive transformations.

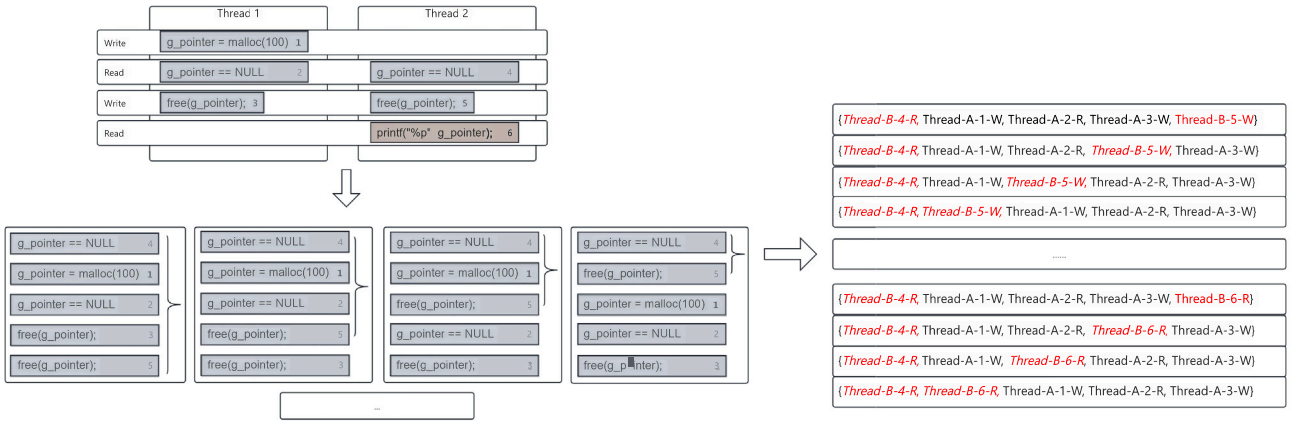

Fig. 7. Example of concurrent behavior window.

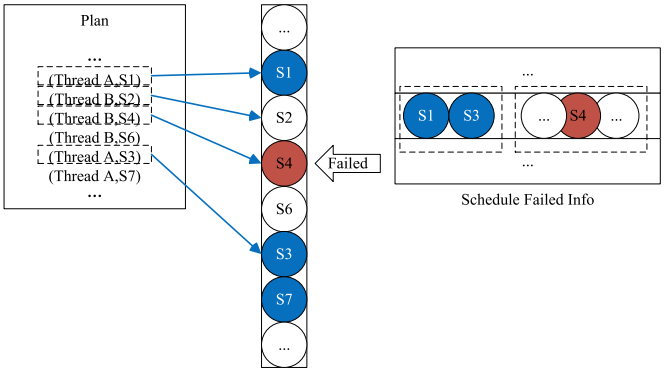

Fig. 8. Demonstration of scheduling plan execution failure results.

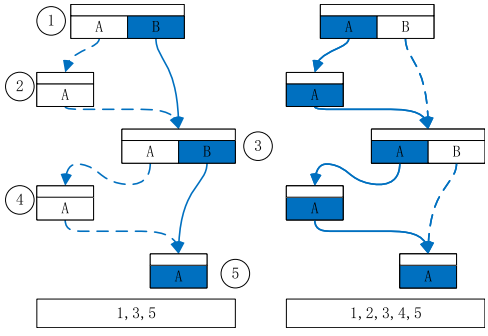

Fig. 9. Using branches to change program execution flow.

This part is divided into assigning a value to the mutation energy through the performance of the seed file, utilizing the difference in the performance of the seed file in different metrics for efficient mutation and controlling the thread scheduling.

5.1. Multi-target optimization mechanisms

At different stages, fuzzy testing should adaptively combine and prioritize bootstrap goals based on the testing scenario. For example, when testing memory-allocated code fragments, seeds related to memory consumption goals should be prioritized; while seed files with shorter distances on the CFG should be preferred to keep program execution close to the target location.

Therefore, it is important to study solutions that balance multiple targets with optimal trade-offs based on multi-objective optimization. This section proposes two optimization strategies: the multi-objective seed energy allocation strategy and the seed mutation optimization strategy.

5.1.1. Energy distribution

TSAFL employs three metrics to identify the running state of programs. However, utilizing the information provided by these metrics poses a challenge. For a comprehensive analysis, multiple metrics should be used to fully leverage the information they offer.

The fuzzing process is divided into two stages: Exploration and Exploitation. During Exploration, seed files are treated equally under different metrics, and an energy balance strategy is used to adjust their energy based on their participation in fuzz testing. Test files are identified after each execution, and seed files with the same code coverage are allocated additional fuzzing time.

As shown in formula (4), the energy $p(i)$ allocated to each seed file q is jointly determined by the parameters $\alpha(i)$ and $S(i)$ preset by AFL, as well as the number of generated input files $f(i)$ that execute the same path, to balance the differences in participation times between different seed files.

$$p(i) = \min\left(\frac{\alpha(i)}{\beta} \cdot \frac{2^{s(i)}}{f(i)}, M\right) \tag{4}$$

After the Exploration, as shown in formula (5), concurrent fuzzy testing uses the concurrent behavior windows in each seed file to calculate the importance coefficient compared to CFG information and code coverage. In formula (5), R_{cc} represents the corresponding coefficient of code coverage, corresponds to the total number of discovered CFG information parameters before Exploration, and corresponds to the total number of discovered concurrent behavior windows.

$$R_{\mathrm{CC}} = \frac{C_C}{C_W} \tag{5}$$

During the exploitation stage, the test tool generates a basic return-on-investment (ROI) model. This means that it treats energy usage as an investment and calculates the corresponding amount of concurrency-related returns. This helps to establish an energy allocation plan.

The energy for each seed file is determined by its code coverage, concurrent behavior window, and CFG measurement information, multiplied by a coefficient. When new seed files are created, their corresponding test files receive a temporary fixed energy reward. The energy of a seed file over time is calculated using formula (6), which affects the file's energy history.

$$\text{energy} = \sum_n R_n \times c_n^{\text{new}} + E_{\text{bonus}} \tag{6}$$

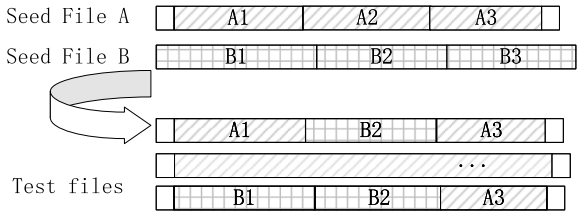

Fig. 10. The mutation method of slicing and randomly reinserting.

Among them, R_n represents the corresponding coefficients of different parameters, and C_n^i represents the number of features of the current seed file on this parameter.

5.1.2. Multi-target mutation strategy

Traditional fuzz testing uses two methods of seed mutation: non-randomness (e.g., flipping bits or performing arithmetic operations on characters) and randomness (e.g., using "havoc" or "splicing" between seed files). However, these methods can be inefficient when dealing with multiple targets. To address this, the MTM algorithm optimizes mutation operations by selecting seeds with advantages in different measurement metrics. This increases the number of high-value seeds and reduces the queue length.

TSAFL creates a pool of candidates for seed files participating in MTM by comparing the differences in metrics of the seed files. For example, statistics are performed according to code coverage and the concurrent behavior windows, and comparisons are made by counting the number of different bits between bitmaps. Two different mutation methods are employed based on AFL's mutation operation: (1) flipping the first two bits of the files after they are spliced back and forth, and (2) slicing two files and randomly re-interspersing them into a new file, which is demonstrated in Fig. 10.

To improve the effectiveness of seed mutation during fuzzy process, in addition to the change of mutation means itself, it is also necessary to pay more attention to which seed files will be added and how to select effective segments more efficiently. In order to measure whether two seed files are worth putting into the mutation process, we calculate the combined value of two seed files with the following formula:

$$Value\langle Seed_A, Seed_B\rangle = \frac{\Phi\langle N_{CFG}, N_{space}\rangle}{Distance\langle Seed_A, Seed_B\rangle} \tag{7}$$

Where the denominator is the distance between the two seed files on the CFG, using the number of hops as a criterion. The numerator is the concurrent space size of the two seed files under (CFG prediction conditions and available execution space), with concurrent behavior window as the counting condition.

We briefly describe the process of seed mutation selection through the code snippet in Fig. 4. As shown in Fig. 11, TSAFL creates a pool of candidate seed files, and groups different seeds for selection through the connection between seed files and specific execution paths. Subsequently, new seed performance evaluation metrics are introduced, and seed files with different concurrent behavior window are selected by the MTM algorithm and put into the seed mutation process. At the same time, we observe the effect of slice fragments on the seed execution effect, calculate the put-in value, and select the slice fragments with higher value.

To reduce the cost of mutation, it is necessary to limit the number of seeds used in the MTM mutation process and the number of MTM module calls. Using too many seeds can consume excessive computing resources and negatively impact the fuzzing process. According to experiments, the MTM seed pool should consist of 15% of all seeds from seed files. Moreover, TSAFL monitors the seed queue's effective change range to balance normal fuzzing with MTM and determine when the MTM module should be called. TSAFL determined that the

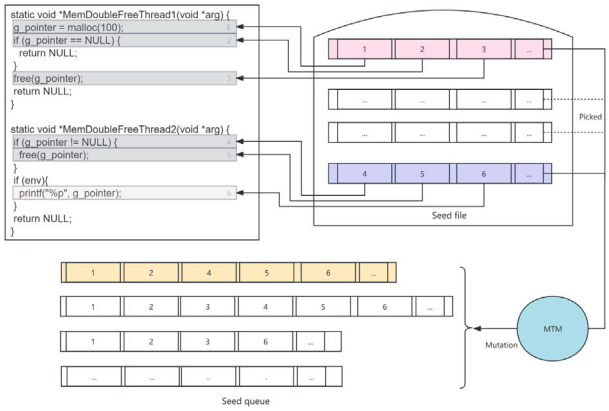

Fig. 11. A simple process of seed mutation selection.

Algorithm 1 MTM algorithm

Input: Initial seeds S, Objectives O
Output: Seed queue Q, Faults F

$Q = S$;
for q in que **do**
  judge(q);
end for
while $curTime < TIMEOUT$ **do**
  if $state == Exploration$ **then**
    energy $= (\bar{E} - \frac{c_q}{c_{\text{all}}}) * E_{\text{Average}}$;
  else*state* $== Exploitation$
    energy $= \sum_n R_n \times c_n^{\text{new}} + E_{\text{bonus}}$;
    for i in $0 \rightarrow energy$ **do**
      $mutation(s)$;
      if $is\_interesting(s^*) == true$ **then**
        $Q = Q + s^*$;
        while in MTM_time **do**
          $crossMutation(s^*)$;
          if $is\_interesting(s^{**}) == true$ **then**
            $Q = Q + s^{**}$;
          end if
        end while
      end if
    end for
  end if
end while

variation range value should be set at 5% after conducting experiments. Algorithm. 1 shows how to integrate multi-objective optimization with the current fuzzing pipeline.

5.2. Thread interleaved scheduling

Multithreaded programs introduce non-deterministic behaviors when different interleavings are involved. Vulnerability detection in these programs requires multiple executions, and there is no guarantee that the corresponding vulnerabilities will be accurately detected. This is because most concurrency vulnerabilities can only be triggered in specific sequences of thread executions.

5.2.1. Period scheduling

Concurrency vulnerabilities can arise in multi-threaded programs due to specific thread interleaving sequences. To address this issue, we

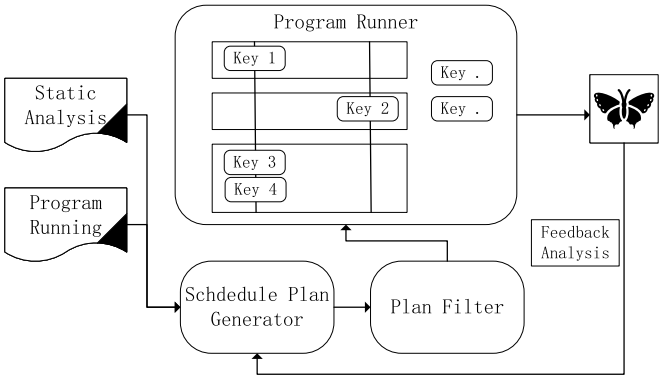

Fig. 12. Schedule process.

propose a mechanism for scheduling threads based on period scheduling. This method partitions program execution into distinct periods according to where concurrency-sensitive behaviors occur and assigns concurrent actions to different periods according to the schedule to precisely control the execution. Using this method, we can simulate the execution of parallel programs as period executions and systematically investigate potential interleaving spaces.

The Fig. 12 demonstrates the period scheduling process, where boxes represent different period, *key** represents different concurrency-sensitive behaviors, and the vertical direction represents different threads. Periodic scheduling relies on Linux system calls and corresponding wrapper instrumentation functions. The internal *Deadline* scheduling scheme of the Linux system assigns a fixed period time to each thread task through *set_sched_dl*, stipulates that all threads are prioritized according to the remaining period time. Meanwhile, *sched_yield* is used to make the thread voluntarily discard all the remaining time in the current period, allowing the thread to enter the next period automatically. In this way, we can adjust thread interleaving from a global perspective, covering the entire program running process.

This method also provides a certain degree of guarantee for the expected scheduling result. Fig. 13 utilizes an example containing `double free` code as a reference, where programs with multiple threads divide program execution into different periodic by *Deadline* according to where concurrency-sensitive behavior occurs. This will distribute the concurrency behavior inside the different periodic according to the plan, which enables high-precision thread interleaving scheduling.

5.2.2. Optimization strategy

Period scheduling is a useful technique for exploring the running state space of multithreaded programs by executing them in a specific order. However, applying this method to fuzzy testing presents challenges due to path explosion. Path explosion can be especially problematic in complex programs, where it arises from the program's complexity and can be difficult to alleviate while ensuring the accuracy of the original method.

To address this issue, researchers have proposed various methods. These include prioritizing specific paths, combining and trimming paths, and eliminating duplicate paths. Given the particularity of concurrent fuzz testing, we have chosen to select and combine different ideas to alleviate the negative impact of path explosion on the testing process. To mitigate this issue, we can use the following strategies:

(1) Remove repeated concurrency-sensitive operation combinations in a single thread.
(2) Use static analysis information to identify unrealizable paths and paths that cannot provide thread interleaving (e.g., create & join to determine whether threads will cycle interleave or not).
(3) Limit the number of threads participating in the scheduling plan at the same time.
(4) Merge threads with the same or similar concurrency behavior.
(5) Use program running information to remove thread planning combinations that do not have program execution space correlation.

There are two methods to mitigate the negative impact of scheduling, but they may result in reduced scheduling precision. Nevertheless, they are necessary to prevent the scheduling tool from breaking and disrupting the entire fuzzing process due to excess schedules. Here are the methods:

- Schedule only basic block (BB) with increased execution times.
- Reduce the number of concurrent sensitive operations participating in scheduling according to certain principles.

This optimization strategy serves two main purposes. Firstly, concurrent fuzz testing requires the program to be actually run to check for defects. Other solutions that rely too heavily on simulation results, such as static analysis, may miss detecting defects. Secondly, this successfully reduced the number of scheduling plans without affecting the overall thread interleaving exploration.

The optimization strategy aims to reduce scheduling schemes that cannot generate new thread interleavings. For example, in software development, code reuse is often used to improve stability, security, and reusability. However, this can lead to repeated fixed concurrent behavior sets in a single thread, which can limit opportunities for thread interleaving. To address this, we limit the number of threads participating in the period scheduling plan to two, allowing us to include all threads in the plan two by two. Fuzz testing can monitor the program's running status without affecting its actual operation. To filter out duplicate paths, we use the concurrent behavior window proposed in this paper as a measure to remove scheduling schemes that cannot generate new concurrent behavior windows. This approach avoids scheduling omissions and can provide an effective way to collect information caused by the execution failure of the scheduling plan.

6. Evaluation

We implemented TSAFL upon SVF, LLVM, and AFL. The static analysis module uses SVF and LLVM to analyze the LLVM-IR code of the source program, which obtains information about key points of concurrency-sensitive behavior. The dynamic collection module relies on the PIN program (Cur-clang), code coverage, and concurrency-sensitive information. The fuzzy testing strategies are based on AFL but differ in the important aspects of thread scheduling, operational analysis, seed selection, and energy allocation. We have conducted thorough experiments to evaluate TSAFL with a set of benchmarks and compared it with various existing techniques. With these experiments, we aim to answer the following research questions:

RQ1. How capable is TSAFL in terms of finding concurrency bugs?

RQ2. How capable is TSAFL of exploring concurrency-related pathways?

RQ3. How does our focus on scheduling contribute to the effectiveness of TSAFL?

RQ4. What runtime overhead is incurred by TSAFL?

6.1. Evaluation setup

6.1.1. Configuration parameters

All our experiments have been performed on the same hardware and software environment, which included an Intel Xeon I7-7700, 32 GB RAM, and docker Ubuntu 18.01 LTS. To reduce the experimental error caused by chance, this paper will use the test tool to run the corresponding tested program for 24 h and repeat the experiment 8 times to take the integer average value as the final result for display and analysis. We make our experimental data and source code via our artifact: https://github.com/JibinArvin/TSAFL

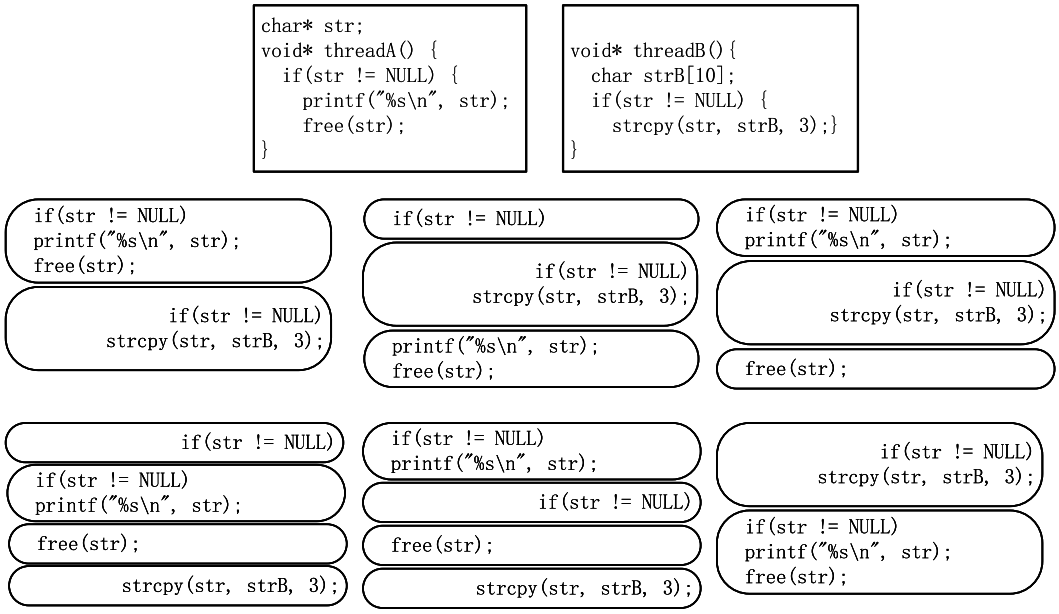

Fig. 13. Double free example scheduling result demonstration.

Table 1
Information about open source programs participating in the experiment.

ID	version	commands	Binary size	web link
pbzip2-c	pbzip2-v1.1.13	pbzip2 -f -k -p4 -S16 @@	312 K	https://github.com/ruanhuabin/pbzip2
pbzip2-d	pbzip2-v1.1.13	pbzip2 -f -k -p4 -S16 -d @@	312 K	https://github.com/ruanhuabin/pbzip2
pigz-c	pigz-2.4	pigz -p 4 -c -b 8 @@	117 k	https://github.com/madler/pigz
vpxdec	libvpx-v1.3.0–5589	vpxdec -t 4 -o /dev/null @@	3.8 M	https://github.com/webmproject/libvpx
convert	ImageMagick-7.0.8–7	convert -limit thread 4 @@ /dev/null	19.4 M	https://imagemagick.org/script/download.php
x264	x264-0.164.r3075	x264 –threads=4 -o /dev/null @@	6.4 M	https://code.videolan.org/videolan/x264.git
xz	XZ-5.3.1alpha	xz –9 -k -T 4 -f @@	182 k	https://github.com/tukaani-project/xz

6.1.2. Benchmark programs

The evaluation dataset consists of the following items:

(1) Parallel compression/decompression utilities including `pbzip2`, `pigz`, `xz`. These tools have been present in GNU/Linux distributions for many years and are integrated into the GNU tar utility.

(2) `vpxdec` is the WebM projects for VP8/VP9. It is used by popular browsers like Chrome, Firefox, and Opera.
(3) `convert(ImageMagick)` is a widely used software suite for displaying, converting, and editing image files.
(4) `x264` is the most established video encoders for H.264/AVC.

All these projects' single-thread functionalities have been intensively tested by mainstream GBFs such as AFL. We try to select the latest stable versions when evaluating software. Table 1 shows the information of the 7 test programs. The first two columns show the program IDs and the benchmark project. `Column3` shows the command line options, where four worker threads are specified to enforce the program to run in multithreaded mode. `Column4` shows the size of the instrumented binary, and `Column5` shows the code location of the test program, which allows direct viewing of the corresponding program.

In addition, to evaluate the effectiveness of our approach, we utilize SCTBench (Sctbench, 2016), a widely used benchmark for concurrent programs. It covers a wide range of bug types, workloads, and applications. Some programs consist of several thousand lines of production C/C++ code. The SCTBench collects 52 concurrency bugs from previous parallel workloads and concurrency testing/verification works. Note that we exclude 16 programs in SCTBench as 5 of them fail to compile on the LLVM platform and the bugs in the other programs can be exposed 100% of the time.

6.1.3. Baselines

We use the following fuzzers during evaluation.

- **AFL++** (Fioraldi et al., 2020) is able to extend the fuzzification process at multiple stages, add new basic blocks to the original code when compiling the code, and can be combined with other techniques.
- **MOPT** (Lyu et al., 2019) is an AFL-based fuzzer that focuses on improving the effect of mutation operations in fuzzing using a custom Particle Swarm Optimization (PSO) algorithm to find the best choice of operator selection probability distribution.

Also, to determine whether our approach is competitive with other approaches in terms of path exploration, we compare with P_{ERIOD} (Wen et al., 2022). P_{ERIOD} is a Controlled Concurrency Testing (CCT) technique designed to detect concurrency bugs by efficiently exploring the space of possible interleavings of concurrent programs through period scheduling.

Unfortunately, the code of the state-of-the-art related works MUZZ (Chen et al., 2020), ConZZER (Jiang et al., 2022) and AutoInter-fuzzing (Ko et al., 2022) are not open source. During the testing phase of TSAFL, we tried to contact the authors of these works, but did not obtain any implementation of either approach. We attempted to replicate some of these methods, but found that the bugs were not fully triggered even in simple test cases. We are not confident of the complete presentation of these works.

Table 2
Statistics of seed files related to 24H vulnerabilities of three tools.

ID	AFL++					MOPT					TSAFL-					TSAFL				
	N_C	N_C^M	N_V^M	N_C^S	N_V^S	N_C	N_C^M	N_V^M	N_C^S	N_V^S	N_C	N_C^M	N_V^M	N_C^S	N_V^S	N_C	N_C^M	N_V^M	N_C^S	N_V^S
pbzip-c	0	0(+3)	0(+1)	0	0	0	0(+3)	0(+1)	0	0	0	0(+3)	0(+1)	0	0	3	3	1	0	0
pbzip-d	0	0(+7)	0(+1)	0	0	0	0(+7)	0(+1)	0	0	0	0(+7)	0(+1)	0	0	7	7	1	0	0
pigz-c	14	14(+7)	1(0)	0	0	10	10(+11)	1(0)	0	0	21	21(0)	1(0)	0	0	21	21	1	0	0
vpxdec	127	93(−28)	2(0)	34	2	94	66(−1)	1(+1)	28	2	182	120(−55)	1(+1)	62	2	108	65	2	43	2
convert	14	3(+4)	1(0)	11	1	12	7(0)	1(0)	5	1	14	7(0)	1(0)	7	1	14	7	1	7	1
x264	0	0(+47)	0(+1)	0	0	0	0(+47)	0(+1)	0	0	0	0(+47)	0(+1)	0	0	47	47	1	0	0
xz	0	0(0)	0(0)	0	0	0	0(0)	0(0)	0	0	0	0(0)	0(0)	0	0	0	0	0	0	0

We evaluate the performance of TSAFL by analyzing its ability to detect concurrent vulnerabilities and its ability to explore paths. We also perform an ablation study of TSAFL without its key components: the number of seeds generated and bugs found. Concurrency defect detection performance refers to the number of program defects related to concurrency that are discovered by fuzz testing tools within the same time period. This is a crucial indicator for evaluating technologies that detect concurrency defects. The analysis of generated sample results evaluates the seed files generated by the fuzz test within a specified time. Seed files are an essential component of the fuzz test process. When the program does not contain defects, the number of seed files and how much concurrency space is covered represents the ability to explore the program internally.

6.2. Bug-finding ability evaluation

Concurrent bug discovery ability refers to the number of seed files related to concurrent bugs discovered and concurrent bugs found by fuzzy testing tools at the same time. It is the most important metric for evaluating fuzzy testing techniques related to concurrent bug discovery.

We tested TSAFL on benchmark programs while comparing it with AFL++ and MOPT. We take the same time budget and use the crash seed that each tool is really able to collect as a standard. Table 2 shows the vulnerability detection results of TSAFL, AFL++ and MOPT.

We denote the total number of Proof-of-Crash (POC) files generated during fuzzing tests as N_C. N_C^M is the number of POC files that can trigger multithreading-relevant defects, and N_C^S is the number of POC files triggering non-multithreading-related defects. Then, we categorize the detected vulnerabilities into concurrent and non-multithreading related vulnerabilities based on their relevance to multithreading, and their numbers are denoted as N_V^M and N_V^S. We mainly focus on N_C^M, N_V^M in Table 2, which can clearly illustrate the fuzzy testing tool's mined ability for concurrency defects.

The importance of N_C^M lies in the ability of concurrent fuzzy testing tools to constantly modify the seed file through mutation operations to find bugs. Obviously, TSAFL has better results compared to other tools in most benchmarking programs (For example, in `pigz-c`, TSAFL generated 21, AFL++ generated 14, and MOPT generated 10). In the `vpxdec` test program, TSAFL finds fewer POC files for multithreading-related defects than AFL++ and MOPT. The reason is that the seed collection conditions is different, and TSAFL retains more "deeper" multithreading related seeds. Although AFL++ and MOPT found more POC files, many of the seeds have mutations that do not trigger crash conditions and do not trigger bugs at runtime. Additionally, TSAFL has been shown to create POC files that trigger concurrency defects that other testing programs fail to find (For example, in `x264`, TSAFL generated 47, while AFL++ and MOPT found none). This reflects that TSAFL's efforts in concurrency awareness, multi-objective optimization, and scheduling can more efficiently approach the location of concurrency defects and successfully generate correlated test files.

x264/common/frame.c (Thread 1)

```
678. void x264_frame_cond_broadcast(
680.     frame->i_lines_completed = i_lines_completed;
```

x264/encoder/analyse.c (Thread 2)

```
3860. completed = h->fref[ l ][ ref >> MB_INTERLACED ]->orig->i_lines_completed;
```

Fig. 14. TSAFL newly detected data race (in x264).

N_V^M is also important, as it represents the ability of the fuzzer to discover program concurrency defects. TSAFL shows a clear advantage in concurrency bug detection, simple data comparisons reveal that TSAFL detects 7 concurrency defects, while AFL++ and MOPT respectively detect 4 and 3. For instance, TSAFL discovers defects that AFL++ and MOPT fail to find, such as 1 concurrency defect in `x264`. We have paid less attention to the roles of N_C, N_C^S, and N_V^S in the analysis process because the detection of concurrent defects is the main focus of this paper. By analyzing the comparative results of N_C^M and N_V^M, we can understand that TSAFL achieves the best results in most of the projects, possesses a better awareness of concurrent programs, and has a better ability to explore multithreaded operating environments.

We use the same time budget to record the specific concurrency bugs discovered and their types, and describe these multithreading related vulnerabilities through Table 3. TSAFL finally reported 7 concurrency bugs, all these 7 concurrency bugs are real and actually triggered at runtime, and thus TSAFL has no false positives in concurrency bugs detection. We manually checked them and 4 of the data races were harmful, they occurred into accessing data structures stored in memory, function pointer calls, and these variables can lead to security issues such as wrong results and data corruption. We reported the 7 concurrency bugs to the relevant developers, 3 of them have been confirmed and we are still waiting for the response from others. We mark the newly detected vulnerabilities (in x264) with a star. As shown in Fig. 14, accesses to variable `i_lines_completed` in `frame.c` and `analyze.c` was not properly synchronized with a lock, leading to data corruption when different threads read it, which could lead to a buffer overflow bug. AFL++ and MOPT were unable to identify a data race that caused a buffer overflow defect in the `x264` program.

In `vpxdec`, TSAFL and AFL++ detected 2 vulnerabilities, and MOPT only detected 1. For the 1 of the vulnerabilities (data race) detected by all tools, TSAFL performs more stable as it detected this bug in all 8 of our experiments, while AFL++ detected it only in 6 and MOPT only in 5 runs (not described in detail in the table).

In `pbzip2`, TSAFL found 2 concurrency vulnerabilities: a data race and a stack-overflow bug. The stack-overflow vulnerability in pbzip2-d is only triggered by a crash when working in multi-threaded mode. In our evaluation, TSAFL detects this vulnerability while other tools fail. This suggests that TSAFL can detect vulnerabilities that are triggered only in multiple threads, even if they do not involve data

Table 3
Real industrial program testing results.

ID	Bug type	AFL++	MOPT	TSAFL-	TSAFL
pbzip2-c	data-race	×	×	×	√
pbzip2-d	stack-overflow	×	×	×	√
pigz-c	dead-lock	√	√	√	√
vpxdec	data-race	√	√	×	√
vpxdec	buffer-overflow	√	×	√	√
convert	data-race	√	√	√	√
*x264	data-race	×	×	×	√

Table 4
Seed files generated in 24H.

ID	AFL++				MOPT				TSAFL-				TSAFL			
	N_{all}	N_{favor}	N_{Cur}	N_{Cur}/N_{all}	N_{all}	N_{favor}	N_{Cur}	N_{Cur}/N_{all}	N_{all}	N_{favor}	N_{Cur}	N_{Cur}/N_{all}	N_{all}	N_{favor}	N_{Cur}	N_{Cur}/N_{all}
pbzip-c	68	3	2	2.94%	51	2	1	1.96%	79	5	3	3.80%	82	5	3	3.67%
pbzip-d	107	6	4	3.74%	98	4	3	3.06%	126	9	5	3.97%	133	9	6	4.51%
pigz-c	151	4	4	2.65%	90	2	2	2.22%	183	7	7	3.83%	183	7	7	3.83%
vpxdec	4299	501	364	8.47%	3352	266	199	5.93%	5394	730	465	8.62%	5286	694	551	10.42%
convert	2457	411	328	13.30%	1425	201	151	10.60%	3164	561	437	13.80%	2190	553	432	19.70%
x264	2122	378	197	9.28%	1619	251	107	6.60%	3152	674	285	9.04%	2566	669	346	13.48%
xz	2070	530	336	16.23%	1812	391	313	17.27%	2277	654	383	15.94%	2203	627	398	18.06%

race. We hypothesize that this is due to the non-stop and random thread scheduling of concurrent programs, which leads to the inability of AFL++ and MOPT to correctly determine the value of the test files based on traditional code coverage, which in turn leads to inefficiencies in the overall process. This proves the necessity of TSAFL, which is also the multiple programs running measurement metrics adopted in this paper. Only in the context of correctly understanding program running can the concurrent program running state space be efficiently explored.

6.3. Pathfinder ability analysis

Seed files are another important indicator to measure the effectiveness of fuzzy testing, and its number represents the ability to explore the program's state space. Table 4 shows the newly generated seed results of each fuzzy test method in 24H. N_{all} is the total number of seed files found throughout the process. N_{favor} is the number of seeds in the favor queue at the end of 24H. N_{Cur} is the number of newly found seed files with new features containing concurrent paths, the larger value of this metric represents that fuzzer can retain more thread interleaving related seeds. We take the average integer value of seed files in 8 experiments to form N_{all}, N_{favor}, N_{Cur}. N_{Cur}/N_{all} determines the probability of selecting a concurrency-relevant seed during seed selection process, which greatly impacts the overall quality of the generated seeds.

From Table 4, the performance of TSAFL still has obvious advantages. First, TSAFL can achieve the highest N_{all} except for the `convert` benchmark, which means that TSAFL maximizes the traversal of program paths and explores the existing execution process. In terms of generating concurrency-relevant seeds (N_{Cur}), TSAFL is more sensitive to the perception of concurrent run information, and performs superiorly among all tested programs. For example, despite the low number of concurrently relevant new features explored by all tools in `pbzip2-d`, TSAFL generated 7 concurrently relevant seeds, which is 3 more than AFL++ (4), and 5 more than MOPT (2). Meanwhile, in relatively large programs (e.g., `convert`), TSAFL still outperforms the exploration ability of other tools, which means that TSAFL can discover more seeds of concurrent paths containing new features.

By comparing the N_{Cur}/N_{all} differences, TSAFL found the concurrent specificity seed to be significantly more advantageous. For example, although AFL++ has the largest N_{all} for `convert` (2457), the value of its N_{Cur}/N_{all} (13.30%) is lower than that of TSAFL (19.70%). Meanwhile, for the test programs (`xz`) that performed well in AFL++ and MOPT, TSAFL can improve it further (18.06%). Also, we can see that N_{favor} outperforms AFL++ and MOPT. For example, in `x264`, TSAFL generates 669 seeds for the favorite queue, which is more than AFL++ (378) and MOPT (251). Therefore, we can conclude that our method contributes to effective seed generation by the concurrent behavior windows, CFG prediction strategy.

To further compare path exploration capabilities, we compared them on SCTBench with the state-of-the-art CCT techniques P_{ERIOD} and Baseline (round after round of execution without going through the schedule at all). We present a cumulative graph in Fig. 15, which shows the number of bugs found by each method as the number of schedules increases. Different colored lines represent different methods. The `X-axis` represents the number of bugs found after x schedule, where the schedule limit is 1000. The `Y-axis` represents the number of concurrent bugs exposed. Overall, TSAFL found more bugs in the same number of schedules. Before 100 schedules, both TSAFL and P_{ERIOD} show excellent results and both methods find more concurrent bugs quickly. After 100 schedules, TSAFL required fewer schedules to find the same number of bugs. For example, over 365 schedules, TSAFL found 46 concurrency bugs, while P_{ERIOD} required 671 schedules. We attribute the effectiveness to our scheduling optimization approach, which is able to combine the results of static and dynamic analysis more efficiently, circumvent ineffective scheduling, and achieve better performance under the same conditions.

6.4. Each module analysis

The core of TSAFL is our scheduling module, the seed energy selection module, and the means of particularized seed mutation. To verify the value of these methods, we disabled these modules in TSAFL: TSAFL- (turning off the scheduling module), TSAFL-E (turning off seed energy selection), and TSAFL-M (turning off the particularized seed mutation means), respectively. We tested these three de-modularization methods together with TSAFL for the 7 programs in Table 1, measuring the number of seeds generated (N_{Seed}) and the number of concurrent bugs triggered (N_{Bug}), and Table 5 shows the results.

Table 5
Comparison results of the TSAFL modules.

ID	TSAFL-M		TSAFL-E		TSAFL-		TSAFL	
	N_{Seed}	N_{Bug}	N_{Seed}	N_{Bug}	N_{Seed}	N_{Bug}	N_{Seed}	N_{Bug}
pbzip-c	73(+9)	1(0)	72(+10)	1(0)	79(+3)	0(+1)	82	1
pbzip-d	107(+26)	1(0)	104(+29)	0(+1)	126(+7)	0(+1)	133	1
pigz-c	165(+18)	1(0)	159(+24)	1(0)	183(0)	1(0)	183	1
vpxdec	4852(+434)	1(+1)	4127(+1159)	1(+1)	5394(−108)	1(+1)	5286	2
convert	1949(+241)	1(0)	1786(+404)	1(0)	3164(−974)	1(0)	2190	1
x264	2464(+102)	1(0)	2283(+283)	1(0)	3152(−586)	0(+1)	2566	1
xz	2079(+124)	0(0)	2144(+59)	0(0)	2277(−74)	0(0)	2203	0

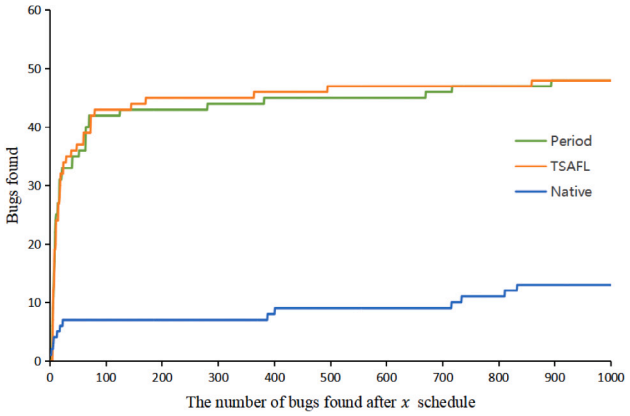

Fig. 15. Total bugs found after scheduling (higher is better).

It can be seen that TSAFL outperforms the other three de-modularization methods in terms of the number of concurrent bugs triggered, and on average, for each program, it finds 14.3%, 28.6%, and 57.1% more concurrent bugs than TSAFL-M, TSAFL-E, and TSAFL-, respectively. As for the number of seeds generated, TSAFL-M generated an average of 1670 seeds, TSAFL-E generated an average of 1525 seeds, TSAFL- generated an average of 2054 seeds, and TSAFL generated an average of 1807 seeds. Thus, TSAFL is able to generate more seeds than TSAFL-M, TSAFL-E in the same amount of time. However, we see that the average result of TSAFL is less than that of TSAFL-, especially in `convert`, where TSAFL- generates 3164 seeds while TSAFL generates only 2190 seeds. In fact, due to the results of the static analysis, it leads to the final generation of too many scheduling policy queues. This slows down the whole test to a great extent. Finding bugs within a limited time is crucial for fuzzy testing, since we cannot run fuzzy tests indefinitely. Therefore, TSAFL- spends most time executing these scheduling results during the same test time, squeezing out other executions and missing some thread interleavings in the program. We can observe that although TSAFL- generates the highest average number of seeds, it misses 4 concurrency bugs found in normal runs. We use scheduling optimization to address this problem by analyzing the results of each execution, correcting some elements that cannot be captured in detail by static analysis, and enhancing the exploration of dynamic analysis to obtain more accurate results. The results demonstrate that our scheduling module, seed energy selection module, and particularized seed mutation means significantly improve the bug-finding ability of our tool to effectively cover infrequent thread interleavings and discover hard-to-find concurrency bugs.

From the results in Table 5, it can be analyzed that the de-scheduling module has the greatest impact on the bug detection aspect. In order to specifically compare the capabilities of the scheduling module, we disabled the scheduling module in TSAFL (TSAFL-) and compared it with AFL++ and MPOT.

We compare the path exploration ability and bug triggering ability of TSAFL-, AFL++ and MOPT. As shown in Table 4, TSAFL- is able to achieve the highest N_{all} and N_{Cur}/N_{all} except for the `xz` benchmark. The average number of newly discovered seed files with new features containing concurrent paths is 227 in TSAFL-, which is higher than AFL++ (177) and MOPT (111). The corresponding average percentage of seeds associated with multithreading is 11.05% in TSAFL-, and 10.99% in AFL++ and 9.20% in MOPT. We analyze the reason for this, which is because in `xz`, TSAFL- spends more time on executing the scheduling results and ignores some concurrent interleavings in the program, hence the introduction of scheduling optimizations is necessary.

In Table 2, even with this module disabled, TSAFL- triggered a higher number of POC files of multithreading-related defects (N_C^M) than in AFL++ and MOPT. (e.g., the average number of POC files that triggered multithreading-related defects, AFL++: 16, MOPT: 12, TSAFL-: 22.) This indicates that we gained more meaningful information in the information collection section by concurrent behavior windows and CFG predictions, and therefore were able to trigger more POC files. However, the results of TSAFL- in discovering concurrency vulnerabilities are on par with AFL++ and MOPT (AFL++: 4, MOPT: 3, TSAFL-: 3), and both are smaller than TSAFL. This also indirectly demonstrates that the scheduling module helps to explore the concurrency dimension and get more meaningful guidance by exploring more intersections of non-stop paths.

6.5. Execution efficiency analysis

It is reliable to measure the execution efficiency of the fuzzy test by the running speed of the tested program, this is because the execution time of the tested program accounts for more than 95% of the fuzzy test process. At the same time, the size of the files involved in running is also an important factor that affects the running speed of the program. In order to achieve uniformity, we selected 5 files of moderate file size in each of the different programs, and ran them repeatedly for 100 times for comparison.

We compared the average execution time of 7 different real-world programs after using AFL++ instrumentation and TSAFL instrumentation, as shown in Table 6. It can be seen that the execution time of the programs after TSAFL instrumentation fluctuates differently depending on the specific program and function calls. The execution time of pbzip2 after TSAFL instrumentation is 637.5% and 366.67% of after AFL++ instrumentation, and the execution time of x264 after TSAFL instrumentation is 176.37% of AFL++ instrumentation. On average, the execution time of all programs after TSAFL instrumentation is 282.33% of after AFL++ instrumentation, while the shorter the overall running time of a program the more its running time is affected.

Overall, the speed of function running after TSAFL instrumentation has significant disadvantage compared to that after AFL++ instrumentation, due to significant differences in information collection methods. The instrumentation function in TSAFL, the tool implemented in this paper, has three parts: code coverage, concurrent sensitive behavior

Table 6
Running speed comparison with AFL++.

ID	AFL++(s)	TSAFL (s)	TSAFL/AFL++
pbzip2-c	1.2	4.4	366.67%
pbzip2-d	0.8	5.1	637.50%
pigz-c	8.2	15.7	191.21%
vpxdec	367.2	431.6	117.54%
convert	2.1	5.3	252.38%
x264	964.5	1701.1	176.37%
xz	3.6	8.45	234.67%

recording, and thread interleaving scheduling, and such collection methods will inevitably have impacts on the program running speed. However, this situation shows a slowing down trend as the program size increases, which is due to the fact that more time is consumed in the computation module in larger programs such as x264, and the percentage of thread concurrency is significantly reduced. Unlike AFL++'s code coverage instrumentation method, TSAFL's coverage instrumentation measures the correlation between the instrumentation position and the concurrent behavior, so programs with smaller overall BB block numbers are more affected. Although the TSAFL instrumentation mode exhibits significant time cost consumption on pbzip2, TSAFL explores the program more in-depth and discovers defects that AFL++ fails to detect. This level of overhead is justified on the basis of a more complex and refined exploration effort, and we will optimize the instrumentation method in future work to reduce the impact of running time.

7. Related work

7.1. Fuzzy testing technology

Fuzzy testing technique (Beaman et al., 2022; Cui et al., 2022) is a widely used testing technique in program testing. It automates testing by using real values as program inputs and randomly exploring different paths of the program. This exploration can be done randomly, with a strategy, or by changing the input based on feedback. While exploring the memory of the program path, the running status of the program is monitored, and various abnormal information about the software is collected, including software crashes, assertion failures, memory leaks, illegal data access, buffer overflows, etc.

Generation-based fuzzing. Generation-based fuzzing tests (Pham et al., 2019; Karamcheti et al., 2018; Böhme et al., 2017) use random input data in a specific format the target expects to generate valuable test files through existing rules that trigger program defects whenever possible. For example, Peach (P, 2020) follows a correct specification or format for inputs, it takes valid input data and performs some transformations by breaking this data into bits and bytes and then fuzzing each bit and byte randomly. The input data structure is maintained but only used to fuzz selected parts. However, generation-based fuzzy testing requires support to generate realistic inputs that match the complexity of the application targeted by the fuzzy test. It would lead to a combinatorial explosion, making it challenging to achieve complete code coverage and limiting its effectiveness in testing complex software systems.

Mutation-based fuzzing. Mutation-based fuzzing (Zalewski, 2020; Lee et al., 2023; Vinesh and Sethumadhavan, 2020; Lyu et al., 2019) has been extensively studied in recent years, which mutates the input data or seed without understanding the format or structure of the data, and constructs new input data by changing certain bits of the normal input to trigger defects. For fuzzy testing of AFL categories, a seed pool containing initial seed files is first needed. Different seed files contain different descriptions for different programs. New test files are created by continuously applying mutation operations to the seed files. After the file-driven program under test is run, the test file is evaluated based on the operation of the program. If the test file shows excellent characteristics in terms of parameters such as code coverage, it is included in the seed pool and then a new round of mutation operations is restarted. The mutation model ensures that the provided input data structure meets the target's expectations. Based on the AFL framework, CollAFL (Gan et al., 2018), TriforceAFL (Pandey et al., 2019) and AFLFast (Böhme et al., 2016) have emerged with different techniques to improve their performance.

Concurrency fuzzing. Concurrency fuzz testing tools attempt use concurrent execution information to explore thread-interleaving (Hong and Kim, 2015; Zhu et al., 2022) in addition to searching the input space of kernels (Jeong et al., 2019) and file systems (Xu et al., 2020). P_{ERIOD} (Wen et al., 2022) treats schedule representations as a sequence of code pieces with thread-sensitive information. RFF (Wolff et al., 2024) employ a biased random search to reduce the search space by exploiting the read-from relation, and guides the search towards under-explored regions of the search space. They focus on the exploration of the scheduling space, thus increasing the diversity of the search space for thread interleaving. TSAFL focuses more on concurrent behavior information to explore more concurrency-related seed files by referencing the concurrent dimension. ConFuzz (Vinesh and Sethumadhavan, 2020) performs fuzz testing of multithreaded applications, leveraging information from source code analysis about the execution paths and thread-related function call sites to rank inputs to detect memory locations where concurrency bugs may occur. ConAFL (Liu et al., 2018) focuses on user-space multithreaded programs, which only detect a subset of vulnerabilities caused by concurrency bugs that cause buffer overflows, double frees, or use-after-free. However, both techniques suffer from scalability issues and are not suitable for testing with large-sized inputs. For example, ConAFL evaluates the biggest binary file of 196k. Our approach mitigates the small test input problem faced by such methods, by concurrently behavior window and cycle-based interleaved scheduling of threads. KRACE (Xu et al., 2020) exploits new concurrency-coverage metric, alias instruction pair, to describe thread interleavings, and uses an evolution algorithm to mutate and generate multi-threaded system call sequences as inputs for concurrency fuzzing. Due to the use the random thread interleaved exploration, it may miss many deep hidden data races that occur only in specific runtime contexts.

7.2. Data race detection

The static analysis (Engler and Ashcraft, 2003; Barbar and Sui, 2021; Voung et al., 2007; Zhao et al., 2024) aims to approximate concurrent programs' behaviors without executing them. For example, LOCKSMITH (Pratikakis et al., 2006) uses a constraint-based technique to detect the data races. FSAM (Sui et al., 2016) suggests a sparse flow-sensitive pointer analysis using a thread-interleaving analysis. Canary (Cai et al., 2021) conducts interference-aware value-flow analysis for checking inter-thread value-flow bugs, achieving both good precision and scalability for millions of lines of code. Due to the lack of knowledge of the actual running of the program, static analysis methods lack information about the context in which the program is running and usually produce false positives. In TSAFL, the static analysis component is currently built on top of SVF and LLVM.

Dynamic analysis (Park et al., 2010; Ahmad et al., 2021) runs a program dynamically through test cases to verify that the program meets requirements or the output meets expectations. Lockset-based methods (Chen et al., 2019; Savage et al., 1997) maintain information about the current locks held by each thread during program execution and report data races when a shared variable is no longer protected by a lock. The happens-before method (Flanagan and Freund, 2009; Li et al., 2019) recognizes the causal relationship between two operations with the help of a logical clock. If the read/write operations on the

same shared variable are concurrent, they are considered to be data races. Hybrid algorithms usually first find suspicious data race based on lockset and then verify the authenticity of the suspicious race with happens-before methods. Due to the uncertainty of thread execution interleavings and the incompleteness of the collected information, the dynamic contention method can generate false positives, while the number of threads interleavings and feasible program paths grows exponentially with time and execution length, resulting in a large execution overhead.

8. Conclusion

This paper presents TSAFL, a new and valuable framework for fuzzing concurrency applications. TSAFL utilizes three innovative thread-aware fuzzing techniques to cover uncommon thread interleavings. These approaches effectively explore the state space of concurrency programs to detect hard-to-find concurrency-related defects. TSAFL was tested on seven user-level applications, and the experimental results demonstrate its effectiveness in exploring interleavings to detect concurrency bugs.

CRediT authorship contribution statement

Jingwen Zhao: Writing – original draft, Software, Methodology, Conceptualization. **Yan Fu:** Validation, Methodology. **Yanxia Wu:** Writing – review & editing, Supervision, Investigation. **Jibin Dong:** Validation, Software. **Ruize Hong:** Software, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to express appreciation for the financial support provided by the Heilongjiang Natural Science Foundation (JJ2019LH2160).

Data availability

Data will be made available on request.

References

Ahmad, A., Lee, S., Fonseca, P., Lee, B., 2021. Kard: Lightweight data race detection with per-thread memory protection. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 647–660.

Arifin, R., Atika, T.R., 2021. Facebook leaks: how does indonesian law regulate it? Ganesha Law Rev. 3 (1), 33–42.

Barbar, M., Sui, Y., 2021. Compacting points-to sets through object clustering. Proc. ACM Program. Lang. 5 (OOPSLA), 1–27.

Beaman, C., Redbourne, M., Mummery, J.D., Hakak, S., 2022. Fuzzing vulnerability discovery techniques: Survey, challenges and future directions. Comput. Secur. 120, 102813.

Böhme, M., Cadar, C., Roychoudhury, A., 2020. Fuzzing: Challenges and reflections. IEEE Softw. 38 (3), 79–86.

Böhme, M., Pham, V.T., Nguyen, M.D., Roychoudhury, A., 2017. Directed greybox fuzzing. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 2329–2344.

Böhme, M., Pham, V.T., Roychoudhury, A., 2016. Coverage-based greybox fuzzing as markov chain. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 1032–1043.

Cai, Y., Yao, P., Zhang, C., 2021. Canary: practical static detection of inter-thread value-flow bugs. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. pp. 1126–1140.

Cai, Y., Zhu, B., Meng, R., Yun, H., He, L., Su, P., Liang, B., 2019. Detecting concurrency memory corruption vulnerabilities. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 706–717.

Chen, Q.L., Bai, J.J., Jiang, Z.M., Lawall, J., Hu, S.M., 2019. Detecting data races caused by inconsistent lock protection in device drivers. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 366–376.

Chen, P., Chen, H., 2018. Angora: Efficient fuzzing by principled search. In: 2018 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 711–725.

Chen, H., Guo, S., Xue, Y., Sui, Y., Zhang, C., Li, Y., Wang, H., Liu, Y., 2020. MUZZ: Thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs. arXiv preprint arXiv:2007.15943.

Chen, D., Jiang, Y., Xu, C., Ma, X., Lu, J., 2018. Testing multithreaded programs via thread speed control. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 15–25.

Cui, L., Cui, J., Hao, Z., Li, L., Ding, Z., Liu, Y., 2022. An empirical study of vulnerability discovery methods over the past ten years. Comput. Secur. 120, 102817.

Engler, D., Ashcraft, K., 2003. Racerx: Effective, static detection of race conditions and deadlocks. ACM SIGOPS Oper. Syst. Rev. 37 (5), 237–252.

Fioraldi, A., Maier, D., Eißfeldt, H., Heuse, M., 2020. {AFL++}: Combining incremental steps of fuzzing research. In: 14th USENIX Workshop on Offensive Technologies. WOOT 20.

Flanagan, C., Freund, S.N., 2009. FastTrack: efficient and precise dynamic race detection. ACM Sigplan Not. 44 (6), 121–133.

Fu, H., Wang, Z., Chen, X., Fan, X., 2018. A systematic survey on automated concurrency bug detection, exposing, avoidance, and fixing techniques. Softw. Qual. J. 26, 855–889.

Gan, S., Zhang, C., Qin, X., Tu, X., Li, K., Pei, Z., Chen, Z., 2018. Collafl: Path sensitive fuzzing. In: 2018 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 679–696.

Hong, S., Kim, M., 2015. A survey of race bug detection techniques for multithreaded programmes. Softw. Test. Verif. Reliab. 25 (3), 191–217.

Inc., G., 2019. Syzkaller is an unsupervised, coverage-guided kernel fuzzer. https://github.com/google/syzkaller.

Jeong, D.R., Kim, K., Shivakumar, B., Lee, B., Shin, I., 2019. Razzer: Finding kernel race bugs through fuzzing. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 754–768.

Jiang, Z.M., Bai, J.J., Lu, K., Hu, S.M., 2022. Context-sensitive and directional concurrency fuzzing for data-race detection. In: Proceedings of the 29th Network and Distributed System Security Symposium. NDSS, p. 1.

Karamcheti, S., Mann, G., Rosenberg, D., 2018. Adaptive grey-box fuzz-testing with thompson sampling. In: Proceedings of the 11th ACM Workshop on Artificial Intelligence and Security. pp. 37–47.

Kidd, N., Reps, T., Dolby, J., Vaziri, M., 2011. Finding concurrency-related bugs using random isolation. Int. J. Softw. Tools Technol. Transfer 13 (6), 495–518.

Ko, Y., Zhu, B., Kim, J., 2022. Fuzzing with automatically controlled interleavings to detect concurrency bugs. J. Syst. Softw. 191, 111379.

Lee, M., Cha, S., Oh, H., 2023. Learning seed-adaptive mutation strategies for greybox fuzzing. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 384–396.

Li, G., Lu, S., Musuvathi, M., Nath, S., Padhye, R., 2019. Efficient scalable thread-safety-violation detection: finding thousands of concurrency bugs during testing. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. pp. 162–180.

Li, J., Zhao, B., Zhang, C., 2018. Fuzzing: a survey. Cybersecurity 1 (1), 1–13.

Liu, C., Zou, D., Luo, P., Zhu, B.B., Jin, H., 2018. A heuristic framework to detect concurrency vulnerabilities. In: Proceedings of the 34th Annual Computer Security Applications Conference. pp. 529–541.

llvm, 2023. llvm-project. https://github.com/llvm/llvm-project.

Lu, S., Park, S., Seo, E., Zhou, Y., 2008. Learning from mistakes: a comprehensive study on real world concurrency bug characteristics. In: Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, pp. 329–339.

Lu, S., Park, S., Zhou, Y., 2011. Finding atomicity-violation bugs through unserializable interleaving testing. IEEE Trans. Softw. Eng. 38 (4), 844–860.

Lyu, C., Ji, S., Zhang, C., Li, Y., Lee, W.H., Song, Y., Beyah, R., 2019. MOPT: Optimized mutation scheduling for fuzzers. In: USENIX Security Symposium. pp. 1949–1966.

Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T., 2020. sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. pp. 778–788.

P, T., 2020. Peach fuzzer. https://www.peach.tech.

Pandey, P., Sarkar, A., Banerjee, A., 2019. Triforce QNX syscall fuzzer. In: 2019 IEEE International Symposium on Software Reliability Engineering Workshops. ISSREW, IEEE, pp. 59–60.

Park, S., Vuduc, R.W., Harrold, M.J., 2010. Falcon: fault localization in concurrent programs. In: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering, vol. 1, pp. 245–254.

Peng, H., Shoshitaishvili, Y., Payer, M., 2018. T-fuzz: fuzzing by program transformation. In: 2018 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 697–710.

Pham, V., Böhme, M., Santosa, A.E., Caciulescu, A.R., Roychoudhury, A., 2019. Smart greybox fuzzing. IEEE Trans. Softw. Eng. http://dx.doi.org/10.1109/TSE.2019.2941681.

Pratikakis, P., Foster, J.S., Hicks, M., 2006. Locksmith: context-sensitive correlation analysis for race detection. Acm Sigplan Not. 41 (6), 320–331.

Rawat, S., Jain, V., Kumar, A., Cojocar, L., Giuffrida, C., Bos, H., 2017. Vuzzer: Application-aware evolutionary fuzzing. In: NDSS, vol. 17, pp. 1–14.

Savage, S., Burrows, M., Nelson, G., Sobalvarro, P., Anderson, T., 1997. Eraser: A dynamic data race detector for multithreaded programs. ACM Trans. Comput. Syst. 15 (4), 391–411.

Sctbench, 2016. Sctbench: a set of c/c++ pthread benchmarks for evaluating concurrency testing techniques. https://github.com/mc-imperial/sctbench.

Sui, Y., Di, P., Xue, J., 2016. Sparse flow-sensitive pointer analysis for multithreaded programs. In: Proceedings of the 2016 International Symposium on Code Generation and Optimization. pp. 160–170.

Sui, Y., Xue, J., 2016. SVF: interprocedural static value-flow analysis in LLVM. In: Proceedings of the 25th International Conference on Compiler Construction. pp. 265–266.

Vinesh, N., Sethumadhavan, M., 2020. Confuzz—a concurrency fuzzer. In: First International Conference on Sustainable Technologies for Computational Intelligence: Proceedings of ICTSCI 2019. Springer, pp. 667–691.

Voung, J.W., Jhala, R., Lerner, S., 2007. RELAY: static race detection on millions of lines of code. In: Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering. pp. 205–214.

Wen, C., He, M., Wu, B., Xu, Z., Qin, S., 2022. Controlled concurrency testing via periodical scheduling. In: Proceedings of the 44th International Conference on Software Engineering. pp. 474–486.

Wolff, D., Shi, Z., Duck, G.J., Mathur, U., Roychoudhury, A., 2024. Greybox fuzzing for concurrency testing. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, pp. 482–498.

Xu, M., Kashyap, S., Zhao, H., Kim, T., 2020. Krace: Data race fuzzing for kernel file systems. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1643–1660.

Yu, M., Lee, J.S., Bae, D.H., 2019. Adaptivelock: efficient hybrid data race detection based on real-world locking patterns. Int. J. Parallel Program. 47 (5–6), 805–837.

Zalewski, M., 2020. American fuzzy lop. https://lcamtuf.coredump.cx/afl/.

Zhao, J., Wu, Y., Dong, J., 2024. Efficient data race detection for interrupt-driven programs via path feasibility analysis. J. Supercomput. 80, 21699–21725.

Zhu, X., Wen, S., Camtepe, S., Xiang, Y., 2022. Fuzzing: a survey for roadmap. ACM Comput. Surv. 54 (11s), 1–36.

Jingwen Zhao is a Ph.D. candidate at the College of Computer Science and Technology, Harbin Engineering University. Her research interests include software security, program analysis, concurrent defect detection, and security requirements.

Yan Fu is a teacher at the College of Computer Science and Technology, Harbin Engineering University. Her research interests are program security, reliability, and concurrency defect analysis.

Yanxia Wu is a professor at the College of Computer Science and Technology, Harbin Engineering University. Her research interests are program analysis, compilation technology, embedded systems and the Internet of Things, and software engineering methods and theoretical development. In these fields, she has published more than 70 papers and, as the project leader, presided over more than 40 scientific research projects.

Jibin Dong is a master candidate at the College of Computer Science and Technology, Harbin Engineering University. His research interests include fuzzy testing, program analysis, and security software development.

Ruize Hong is a Ph.D. candidate at the College of Computer Science and Technology, Harbin Engineering University. His research interests include compiler optimization, embedded software and program security analysis.